

Data Science for Software Engineering

Assignment 1 Report

Hadoop YARN - Architectural Recovery Using Clustering Algorithms and LLM-Based Semantic Summarization



Professor: Dr. Mohamed Soliman

Group 10

Sujit Jagdishbhai Maniya (4038595)

Mihir Sane (4086665)

Aryan Solanki (4106036)

Jenish Jai (4090405)

Indudhara S (4106839)

1 Introduction

Apache Hadoop YARN (Yet Another Resource Negotiator) serves as the core cluster management layer for the Hadoop ecosystem [1]. It is responsible for managing computing resources and scheduling applications across distributed nodes. This study focuses specifically on the **ResourceManager Scheduler Capacity** component, located within the `org.apache.hadoop.yarn.server.resourcemanager.scheduler.capacity` package. As the ResourceManager is the central authority for resource arbitration, the Capacity Scheduler plays a vital role in enabling multi-tenancy, ensuring that system resources are shared efficiently among multiple organizations while maintaining guaranteed capacities.

The primary motivation for this research stems from the challenge of **architectural drift**. Large-scale software systems like Hadoop YARN undergo continuous evolution, which often leads to a divergence between the originally intended design and the actual implementation. This drift complicates system maintenance, increases technical debt, and obscures the logical boundaries between components [2]. By performing architectural recovery, we aim to extract high-level structures from the source code to evaluate the current modularity of the system. This project specifically investigates how automated clustering algorithms can reconstruct these boundaries and how emerging Large Language Model (LLM) technologies can assist in interpreting and documenting the resulting architecture.

1.1 Background

1.1.1 Software Clustering and Structural Analysis

Architectural recovery typically utilizes software clustering to partition a system's classes into cohesive subsystems. This study leverages the **ARCADE** tool suite to implement and compare three foundational algorithms [3]:

- **ACDC (Algorithm for Comprehension-Driven Clustering)**: A structural clustering algorithm that groups classes based on common subsystem patterns, such as "support" or "central" components, to mimic human architectural reasoning [4].
- **WCA (Weighted Conglomerative Algorithm)**: A hierarchical clustering approach that uses similarity measures like the Unweighted External Measure (UEM) to group entities based on shared dependencies.
- **LIMBO (Likelihood Based Optimization)**: An information-theoretic algorithm that treats clustering as a data compression problem, aiming to minimize the loss of mutual information (IL) between software entities [5].

Evaluating these algorithms requires objective metrics. The **a2a (Architecture-to-Architecture)** metric is used to determine structural similarity between outputs, while the **Cvg (Coverage)** metric measures the degree of overlap between cluster boundaries. Literature indicates that different algorithms often capture distinct "views" of an architecture [6], necessitating a comparative approach.

1.1.2 LLMs in Architectural Recovery

Recent research has begun exploring Large Language Models (LLMs) to provide the semantic context that structural clustering lacks [7]. While clustering identifies groups, LLM like **nvidia/Llama-3.1-Nemotron-70B-Instruct-HF** can analyze the underlying source code to generate human-readable titles and summaries [8]. This integration represents a shift toward "semantic architectural recovery," where AI-driven qualitative analysis complements traditional quantitative structural data to provide a holistic view of software systems.

The following section presents the study design and methodology to investigate these research questions.

2 Study Design

The study aims to address the following questions regarding the intersection of structural analysis and artificial intelligence:

- **RQ1:** How do different clustering algorithms vary in their ability to accurately determine the architectural components of a system?
- **RQ2:** How do different prompting techniques impact LLM’s ability to accurately describe architectural components based on source code?

2.1 Research Methodology and Process

The research focuses on recovering and analyzing the software architecture of a selected subsystem from the Apache Hadoop project. The chosen subsystem is the Hadoop YARN ResourceManager Capacity Scheduler, which plays a critical role in resource allocation and job scheduling within distributed computing environments. Due to its complex structure and extensive interactions among components, the Capacity Scheduler provides an appropriate case study for evaluating architecture recovery techniques based on clustering algorithms and Large Language Models (LLMs).

2.1.1 Dependency Extraction and Filtering

The initial step involved compiling the Hadoop YARN ResourceManager module to generate the compiled `.jar` file. Using ARCADE JavaParser with `MASTER_PREFIX=org.apache.hadoop`, we generated a Hadoop-only master RSF file with 11,972 dependency lines. We then filtered this file for the `org.apache.hadoop.yarn.server.resourcemanager.scheduler.capacity` package, producing a focused RSF file with 1,812 dependency lines. This newer pipeline avoids clustering external Java, Jackson, SLF4J, and other library classes.

2.1.2 Multi-Algorithm Clustering

To compare different architectural "perspectives," we applied three distinct clustering algorithms to our focused dataset:

1. **ACDC:** A pattern-driven structural clustering tool.
2. **WCA:** A hierarchical algorithm utilizing the Unweighted External Measure (UEM) similarity measure.
3. **LIMBO:** An information-theoretic algorithm utilizing the Information Loss (IL) similarity measure.

Each clustering output now contains 466 containment lines, representing the Hadoop classes retained after the cleaner package-filtered dependency extraction. ACDC produced 7 clusters, while WCA and LIMBO each produced 50 clusters.

2.1.3 Evaluation Metrics

To assess the quality of the recovered architectures, two architecture comparison metrics provided by the ARCADE framework were utilized.

Architecture-to-Architecture (A2A)

The A2A metric measures the structural similarity between two architectural decompositions. It quantifies the extent to which clusters generated by different algorithms contain similar software entities. Higher A2A values indicate stronger agreement between architectural views.

Coverage (CVG)

The Coverage metric evaluates how much of one architecture is represented within another architecture. It provides an indication of the completeness and overlap between two clustering solutions.

These metrics were calculated for all pairs of clustering algorithms, including ACDC, LIMBO, WCA, and ARC. The resulting values were analyzed to identify similarities, differences, and trends among the recovered architectures.

In addition to quantitative evaluation, qualitative assessment was conducted on the architectural descriptions generated by the LLMs. The quality of the generated summaries was evaluated based on their ability to accurately capture component responsibilities, interactions, and high-level architectural concepts.

2.1.4 ARC Semantic-Structural Clustering Implementation

We implemented ARC clustering to combine semantic source-code embeddings with structural dependency data. We utilized the **jinaai/jina-code-embeddings-0.5b** model with 4-bit quantization to generate semantic vectors from the Hadoop YARN Capacity Scheduler Java files, calculating similarity via cosine distance. Structural matrices were simultaneously built from the RSF class dependencies. These matrices were fused using $\text{ALPHA} = 0.5$ to balance both architectural facets equally. Finally, we applied Agglomerative Clustering to group the components into 10 target clusters.

Before interpreting the A2A and CVG values, it is important to consider whether the compared architectures use the same entity set and granularity. In our case, ARC produced 99 containment relations distributed across 10 clusters, while the structural output contained 466 containment relations. This suggests that the outputs may represent the system at different levels of detail. Therefore, A2A/CVG comparisons should be interpreted cautiously unless the architectures are normalized to the same set of entities, such as the same Java classes/files. Otherwise, the metric values may reflect differences in representation rather than only differences in architectural quality. With this limitation in mind, the ARC output was compared against the ACDC, WCA, and LIMBO clustering outputs using the same ARCADE A2A and Cvg metrics. This allowed us to evaluate how the ARC semantic-structural clustering result differs from the traditional structural clustering outputs, while acknowledging that differences in entity granularity may influence the metric values.

2.1.5 Hierarchical Summarization Pipeline

The distinct multi-level summarization pipelines were developed to recover architectural understanding from source code.

Directory-Based Hierarchical Summarization

The hierarchical summarization pipeline successfully completed all 99 leaf-level summaries and produced the final text file as the root-level architectural overview.

At the leaf level, flat prompts generated technically correct but shallow descriptions, mostly summarizing method signatures and variable names without connecting the class to any broader purpose. Contextual hierarchical prompts produced noticeably richer output for example, summaries for classes inside `allocator` and `preemption` subdirectories clearly described their role within the Capacity Scheduler rather than just listing what methods do.

At the branch level, this difference became more visible. Branch summaries built from contextual prompts described relationships between classes within a subsystem, while those from flat prompts largely repeated class-level details without drawing higher-level connections.

The final root summary reflected this gap clearly the contextual approach produced a coherent overview of how the Capacity Scheduler manages queue hierarchies, resource limits, and tenant-level scheduling, while the flat approach produced a loosely connected list of component descriptions.

Cluster-Based Hierarchical Summarization

The cluster-based hierarchical summarization pipeline successfully generated architectural summaries for all clusters recovered by the ARC, ACDC, and LIMBO clustering algorithms. Unlike the directory-based approach, this method organized software entities according to architecture recovery results, allowing the summarization process to focus on logically related components rather than physical source-code locations.

At the file level, individual Java classes were first summarized to capture their primary responsibilities, core logic, inputs and outputs, and dependencies. These file-level summaries were then aggregated according to their assigned clusters and used as contextual input for a second-stage summarization process.

At the cluster level, the generated summaries synthesized the responsibilities of multiple related classes into higher-level architectural descriptions. The summaries consistently identified the major components within each cluster, explained how those components interact, and described their overall role within the Hadoop YARN Capacity Scheduler architecture.

The final outputs provided architecture-centric descriptions for the recovered ARC, ACDC, and LIMBO clusters, offering a higher-level view of the Capacity Scheduler structure and complementing the structural information produced by the clustering algorithms. This approach enabled the recovery of both architectural organization and semantic understanding, producing summaries that were more suitable for architecture documentation and system comprehension than isolated file-level descriptions.

The results obtained from applying these methods are presented in the next section.

3 Results

The results obtained from the applied methods provide an initial understanding of the structural patterns within the Hadoop YARN Capacity Scheduler component, focusing on how different algorithms group the classes and the structural similarity of their recovered architectures.

3.1 RQ1: Clustering Algorithm Comparison

Evaluation via ARCADE metrics reveals that ACDC and WCA produced the most structurally similar architectures with an A2A score of 88.391, followed by WCA vs. Limbo (85.794) and Limbo vs. ACDC (83.342). However, coverage (Cvg) values are predominantly 0.0, indicating almost no direct class-level overlap between the recovered clusters across different algorithms. The sole exception is an asymmetric, minimal overlap between ACDC and WCA (0.1429 / 0.02). This indicates that while ACDC and WCA exhibit a high degree of similarity in their overall architectural structure, all three algorithms still establish significantly different component boundaries.

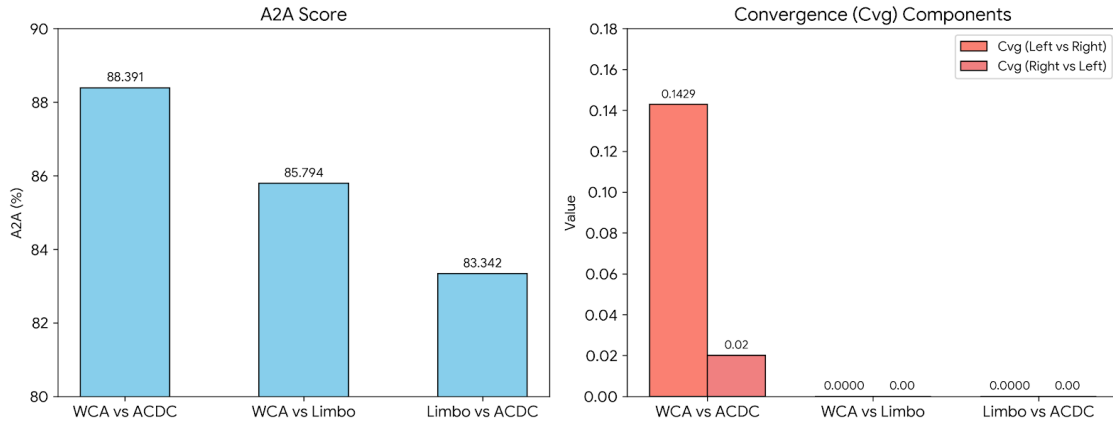


Figure 1: Pairwise Comparison of Clustering Algorithms

The ARC clustering output was compared against the Week 1 structural clustering outputs. ARC produced 99 containment lines across 10 clusters. The A2A scores between ARC and the traditional outputs were substantially lower: ARC vs. ACDC scored 26.520, ARC vs. Limbo scored 26.227, and ARC vs. WCA scored 26.061. These lower scores indicate that ARC produces a significantly different architectural view, driven by the integration of semantic source-code embeddings with structural dependency data.

The Coverage results for ARC were similarly low. ARC vs. ACDC and ARC vs. WCA yielded 0.0 Coverage in both directions, while ARC vs. Limbo showed a minor asymmetric overlap of 0.1 from ARC to Limbo and 0.02 from Limbo to ARC. These metrics reinforce that ARC captures an entirely different architectural perspective of the Capacity Scheduler component compared to traditional structural algorithms.

The results show that structural and semantic recovery approaches produced different architectural views of the Hadoop YARN Capacity Scheduler.

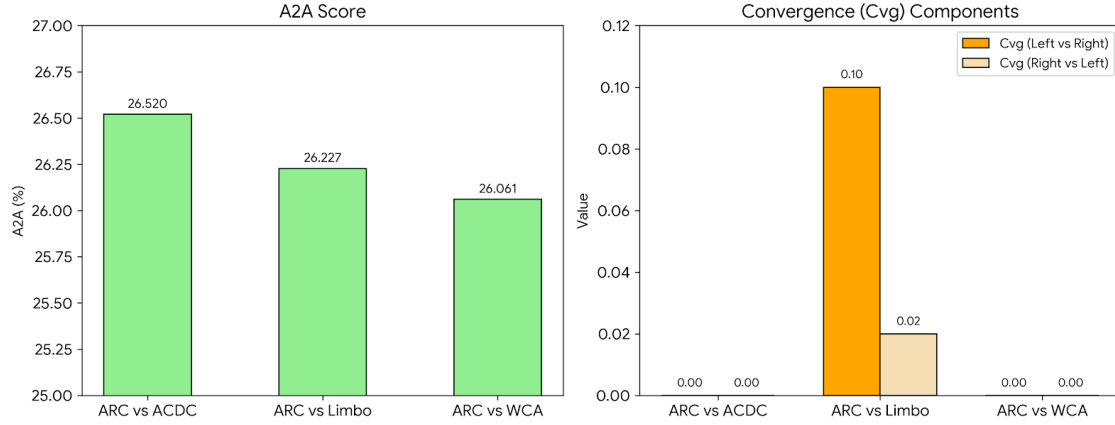


Figure 2: Pairwise Comparison of Clustering Algorithms

Structural vs Semantic Recovery Findings

The structural approaches, ACDC, LIMBO, and WCA, mainly reflected implementation-level relationships. ACDC produced broad architectural groups such as configuration management, placement processing, queue ordering, dynamic queue management, container allocation, and the main Capacity Scheduler subsystem. However, some clusters were very large, which reduced the level of detail. LIMBO produced more fine-grained clusters and identified smaller responsibilities such as metrics, capacity calculation, resource rounding, configuration stores, and preemption, but many clusters were very small, making the architecture appear fragmented. WCA was less useful because its output was highly imbalanced, with most entities grouped into one large cluster.

The semantic/hybrid approach, ARC, produced the most interpretable result. ARC generated 10 clusters and recovered meaningful conceptual subsystems such as the Capacity Scheduler core, queue management, capacity calculation, configuration management, placement rules, preemption, queue capacity updates, and node label handling. These clusters were more balanced and aligned better with architectural responsibilities rather than only source-code dependencies.

The comparison shows that structural recovery is useful for identifying implementation-based relationships, but it can produce clusters that are either too broad or too fragmented. Semantic/hybrid recovery is stronger for identifying conceptual responsibilities and grouping files that serve similar architectural purposes. In this case, ARC provided the clearest architectural recovery because it captured the functional meaning of components more effectively than the purely structural approaches.

Overall, the findings suggest that semantic/hybrid recovery gave a better high-level understanding of the Capacity Scheduler architecture, while structural recovery provided useful supporting evidence about dependency-based organization. Together, the results show that the system is organized around queue-based resource management, supported by capacity calculation, container allocation, configuration management, placement rules, dynamic queue updates, monitoring, and preemption.

Algorithm	Analysis Type	Main Information Used	Output / Result
ACDC	Structural	Dependency relationships and implementation structure	Broad architectural subsystems
LIMBO	Structural	Dependency similarity patterns	Fine-grained dependency-based clusters
WCA	Structural	Weighted dependency relationships	Structural clusters, but highly imbalanced in this result
ARC	Semantic / Hybrid	Semantic similarity with structural support	More balanced and architecturally meaningful clusters

Table 1: Comparison of Structural and Semantic Recovery Results

3.2 RQ2: Prompting Effects on Architecture Comprehension

The directory-based result recovered a package-level view of the Hadoop YARN Capacity Scheduler architecture. It presents the system as a set of source-code submodules: `top_level`, `allocator`, `conf`, `placement`, `policy`, `preemption`, and `queuemanagement`. This result shows that the Capacity Scheduler is organized around resource scheduling, container allocation, configuration management, placement decisions, queue ordering policies, preemption, and dynamic queue management.

The cluster-based result recovered a more conceptual architectural view. Instead of only following package boundaries, it identified functional subsystems such as core scheduling and resource allocation, queue hierarchy management, capacity calculation, container allocation, configuration persistence, dynamic queue management, placement rule management, metrics, user limit management, and preemption.

Aspect	Directory-Based Result	Cluster-Based Result
Main architectural view	Package-level structure	Conceptual subsystem structure
Main recovered units	<code>top_level</code> , <code>allocator</code> , <code>conf</code> , <code>placement</code> , <code>policy</code> , <code>preemption</code> , <code>queuemanagement</code>	Scheduling, queue management, capacity calculation, configuration persistence, placement, allocation, metrics, preemption
Granularity	Coarser and easier to read	More detailed and analytical
Strongest result	Clear source-tree organization	Clear functional architecture
Limitation	Less detail about cross-cutting concerns	Some clusters are too broad or too fine-grained depending on the algorithm

Table 2: Comparison of Directory-Based and Cluster-Based Architecture Recovery Results

The directory-based result clearly identifies the main implementation areas of the Capacity Scheduler. The `top_level` component acts as the core scheduler, coordinating resource management and scheduling. The `allocator` component handles container allocation and deallocation. The `conf` component manages configuration storage, validation, access control, and versioning. The `placement` component supports placement decisions. The `policy` component defines queue ordering behavior. The `preemption` component manages resource reclamation by terminating lower-priority containers. The `queuemanagement` component manages auto-created leaf queues and dynamic capacity adjustment.

The cluster-based result provides a richer architectural interpretation. It separates concerns that are not always visible from the directory structure alone. For example, it identifies a capacity calculation subsystem containing absolute, percentage, weight-based, and root capacity calculators. It also highlights queue capacity updates, node label handling, user weights, queue metrics, configuration persistence implementations, placement rule conversion, and preemption management as distinct architectural responsibilities.

The directory-based result is stronger for explaining how the codebase is physically organized. It gives a straightforward overview of the major packages and their responsibilities. However, it does not expose some finer architectural concerns because many related responsibilities are mixed within the same package.

The cluster-based result is stronger for architecture recovery because it reveals hidden or cross-cutting functional groupings. It shows that the Capacity Scheduler architecture is not only divided by packages, but also by responsibilities such as resource calculation, runtime queue adaptation, configuration persistence, metrics, and preemption.

Overall, both results are complementary. The directory-based result presents the source-tree architecture, while the cluster-based result presents the recovered functional architecture. Together, they show that the Hadoop YARN Capacity Scheduler is organized around queue-based resource management, supported by container allocation, configuration management, placement rules, queue ordering policies, dynamic queue updates, capacity calculation, monitoring, and preemption.

The following section discusses these findings in greater detail, interpreting their implications for architectural recovery.

4 Discussion

To address the first research question, we interpret the observed clustering outcomes in relation to architectural accuracy and component identification. The following discussion examines how effectively the applied algorithms capture meaningful structural boundaries within the Hadoop Yarn system.

4.1 RQ1: Clustering Algorithm Comparison

4.1.1 Architectural Isolation of the Capacity Scheduler

A significant outcome of the extraction phase was the reduction in the size of the dependency graph following the application of the Hadoop project-level package filter. Using `MASTER_PREFIX=org.apache.hadoop`, the master RSF contained 11,972 dependency lines. After filtering for the `org.apache.hadoop.yarn.server.resourcemanager.scheduler.capacity` package, the focused RSF contained 1,812 dependency lines.

This reduction indicates that the Capacity Scheduler represents a well-defined subsystem within the larger ResourceManager module. The refined extraction process retains Hadoop project classes while excluding external Java, Jackson, SLF4J, and other third-party library dependencies from the clustering dataset. This filtering step is essential because the objective of the study is to recover the architecture of Hadoop YARN rather than the architecture of external libraries.

Although the focused RSF is substantially smaller, the remaining 1,812 dependency lines demonstrate that the Capacity Scheduler maintains considerable internal complexity. Its classes interact across queue hierarchies, resource limits, scheduling policies, and tenant-level capacity management. These observations suggest that the subsystem possesses a clearly defined external boundary while maintaining a rich and highly interconnected internal structure.

4.1.2 Interpretation of Identical Containment Counts

After applying ACDC, WCA, and Limbo to the focused RSF, each algorithm produced 466 containment lines. At first glance, this observation may suggest that the algorithms produced identical results; however, the identical containment count merely indicates that the same set of input classes was clustered by each algorithm.

The resulting cluster structures differ substantially. ACDC produced 7 clusters, whereas WCA and Limbo each generated 50 clusters. This indicates that the algorithms generated different architectural decompositions despite operating on the same dataset. The containment count reflects the number of class-to-cluster assignments but does not provide information regarding how classes are grouped within those clusters.

Consequently, meaningful comparison requires the use of the A2A and Coverage metrics, which provide insights into the similarity and differences among the recovered architectural views.

4.1.3 Interpretation of A2A and Coverage Results

The updated A2A results indicate that ACDC and WCA form the most structurally similar pair, with an A2A score of 88.391. Limbo and WCA follow with a score of 85.794, while ACDC and Limbo achieve a score of 83.342. These scores indicate that all three algorithms recover architectures exhibiting a degree of structural similarity, with ACDC and WCA producing the most comparable architectural decompositions.

The Coverage results provide a more detailed perspective on class-level overlap. Most Coverage values remain 0.0, indicating that the algorithms frequently disagree regarding the exact composition of individual clusters. However, ACDC and WCA exhibit limited overlap. Specifically, ACDC-to-WCA achieved a Coverage score of 0.1429, whereas WCA-to-ACDC achieved a score of 0.02. This indicates the presence of limited and asymmetric overlap between the two clustering solutions.

The distinction between the A2A and Coverage metrics is significant. A2A evaluates similarity at the architectural level by comparing overall structural organization, whereas Coverage assesses whether the same software entities are assigned to corresponding clusters. Consequently, two clustering solutions may exhibit similar architectural structures while differing substantially in the allocation of individual classes.

4.1.4 ARC Clustering and Semantic–Structural Comparison

ARC clustering was also applied by combining semantic similarity derived from source code embeddings with structural similarity extracted from the RSF dependency data. In contrast to ACDC, WCA, and Limbo, which primarily rely on structural dependency information, ARC incorporates semantic information derived from source code embeddings.

The ARC output produced 99 containment lines distributed across 10 clusters. When compared with the clustering outputs generated by the structural algorithms, substantially lower A2A scores were observed. Specifically, ARC versus ACDC achieved an A2A score of 26.520, ARC versus Limbo achieved 26.227, and ARC versus WCA achieved 26.061. These low scores indicate that ARC produced a substantially different architectural decomposition from the traditional ARCADE clustering algorithms.

The Coverage results were similarly low. ARC versus ACDC and ARC versus WCA produced Coverage values of 0.0 in both directions. ARC versus Limbo exhibited a small degree of overlap, with Coverage values of 0.1 from ARC to Limbo and 0.02 from Limbo to ARC.

These findings suggest that ARC captures an alternative perspective of the Capacity Scheduler architecture. By integrating semantic information extracted from source code with structural dependency information, ARC emphasizes conceptual and functional relationships that may not be apparent in purely dependency-based clustering approaches.

4.2 RQ2: Prompting Effects on Architecture Comprehension

The comparison between the directory-based and cluster-based results shows that the choice of architectural context has a strong influence on the type of architecture

recovered. The directory-based result reflects the physical organization of the source code, while the cluster-based result reflects a more conceptual organization based on functional responsibilities. This distinction is important because software architecture is not always visible directly from package structure alone.

The directory-based result was useful because it produced a clear and understandable overview of the Capacity Scheduler’s implementation structure. By following the existing source tree, it identified major submodules such as `allocator`, `conf`, `placement`, `policy`, `preemption`, and `queuemangement`. This makes the result easy to interpret and useful for developers who want to understand how the codebase is organized. However, this approach is limited because it assumes that the package structure accurately represents the architecture.

The cluster-based result addressed this limitation by recovering functional groupings that were not always obvious from the directory structure. For example, it identified capacity calculation, queue capacity updates, node label handling, user weights, queue metrics, configuration persistence, placement rule conversion, and preemption management as distinct architectural responsibilities. These findings suggest that cluster-based recovery can reveal hidden or cross-cutting concerns that are difficult to identify from package boundaries alone.

At the same time, the cluster-based result also has limitations. Some clustering algorithms produced groups that were either too broad or too fine-grained. For example, ACDC produced one large cluster containing many Capacity Scheduler responsibilities, which made it harder to separate queue management, capacity calculation, and preemption as independent architectural concerns. The LLM summaries were useful in this respect because they transformed raw clusters into readable architectural descriptions, making it easier to understand the purpose of each recovered group.

The results also show that directory-based and cluster-based recovery should not be treated as competing approaches. Instead, they provide complementary views of the same system. The directory-based result explains how the Capacity Scheduler is physically organized in the source tree, while the cluster-based result explains how the system is functionally organized around architectural responsibilities.

Overall, the findings suggest that effective architecture recovery benefits from both structural context and functional interpretation. Directory-based summarization is useful for recovering the source-level organization of a system, but cluster-based summarization provides stronger support for identifying conceptual subsystems. For the Hadoop YARN Capacity Scheduler, this combined view shows that the architecture is centered on queue-based resource management and supported by capacity calculation, container allocation, configuration management, placement rules, queue ordering, dynamic queue updates, monitoring, and preemption.

These results indicate that LLM-assisted architecture recovery is most effective when the model is given structured context rather than isolated files alone. Hierarchical and cluster-based inputs help the LLM move beyond local code descriptions and produce higher-level architectural explanations. This supports the broader conclusion that prompt structure and input organization play an important role in the accuracy and usefulness of LLM-generated architectural descriptions.

The final section summarizes our key findings and outlines directions for future work.

5 Threats to Validity

Although our results provide a solid baseline, it is important to acknowledge the limitations and potential sources of errors that can affect the validity of our findings. These are categorized into three areas:

5.1 Construct Validity

Construct validity concerns whether the selected methods and evaluation metrics accurately measure architectural similarity and recovery quality. In this study, A2A and Coverage (CVG) metrics were used to compare the architectures generated by ACDC, LIMBO, WCA, and ARC. However, differences in clustering granularity and decomposition strategies may influence the metric values. For example, ACDC produced a small number of high-level clusters, whereas LIMBO generated a larger number of fine-grained clusters. Consequently, the metrics may capture differences in clustering structure rather than purely reflecting architectural quality. Additionally, the absence of a ground-truth architecture for the Capacity Scheduler subsystem limits the ability to verify the absolute correctness of the recovered architectures.

5.2 External Validity

This threat concerns whether our findings can be applied beyond our specific study context. We focused exclusively on the Hadoop YARN ResourceManager Capacity Scheduler. Because Hadoop is a mature, highly decoupled system, the clustering behavior we observed may not be applicable for smaller and monolithic projects. Our results are a snapshot of one component, not a universal rule for all software.

5.3 Reliability

The ARCADE-based clustering results are largely reproducible because the commands and RSF inputs are deterministic. However, reliability is affected by the LLM-based summarization, since small changes in prompts, model settings, or generation behavior may produce different architectural descriptions. Another concern is the preprocessing pipeline, including RSF filtering, cluster normalization, and mapping raw entities back to Java files; changes in these steps could affect skipped entries, file assignments, and final summaries. To reduce this threat, the same scripts, prompts, clustering outputs, and artifacts were used consistently, although the probabilistic nature of LLM generation means that exact wording may vary across runs.

6 Conclusion and Future Work

This report shows that architecture recovery for the Hadoop YARN Capacity Scheduler benefits from combining clustering techniques with LLM-based summarization. The clustering results showed that different algorithms recover different architectural views: ACDC produced broad structural groupings, LIMBO produced more fine-grained clusters, WCA was highly imbalanced, and ARC produced the most interpretable semantic/hybrid decomposition. The LLM summaries made these clusters easier to understand by translating raw file groupings into architectural responsibilities such as queue management, capacity calculation, container allocation, configuration management, placement rules, monitoring, and preemption. The directory-based and cluster-based summaries were also complementary: the directory-based approach explained the source-tree organization, while the cluster-based approach revealed functional and cross-cutting architectural concerns.

Future work should focus on evaluating the recovered architecture against expert-written documentation or developer judgments to better measure its correctness. Further experiments could also compare more models, refine prompts, test larger Hadoop components, and investigate why coverage scores remain low even when A2A suggests architectural similarity. This would help make LLM-assisted architecture recovery more reliable, explainable, and useful for large software systems.

References

- [1] V. K. Vavilapalli, A. C. Murthy, C. Douglas, S. Agarwal, M. Konar, R. Evans, T. Graves, J. Lowe, H. Shah, S. Seth, *et al.*, “Apache hadoop yarn: Yet another resource negotiator,” in *Proceedings of the 4th annual Symposium on Cloud Computing*, pp. 1–16, 2013.
- [2] D. E. Perry and A. L. Wolf, “Foundations for the study of software architecture,” *ACM SIGSOFT Software engineering notes*, vol. 17, no. 4, pp. 40–52, 1992.
- [3] M. Schmitt Laser, N. Medvidovic, D. M. Le, and J. Garcia, “Arcade: an extensible workbench for architecture recovery, change, and decay evaluation,” in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 1546–1550, 2020.
- [4] V. Tzerpos and R. C. Holt, “Acdc: an algorithm for comprehension-driven clustering,” in *Proceedings Seventh Working Conference on Reverse Engineering*, pp. 258–267, IEEE, 2000.
- [5] P. Andritsos, P. Tsaparas, R. J. Miller, and K. C. Sevcik, “Limbo: Scalable clustering of categorical data,” in *International Conference on Extending Database Technology*, pp. 123–146, Springer, 2004.
- [6] T. Lutellier, D. Chollak, J. Garcia, L. Tan, D. Rayside, N. Medvidović, and R. Kroeger, “Measuring the impact of code dependencies on software architecture recovery techniques,” *IEEE Transactions on Software Engineering*, vol. 44, no. 2, pp. 159–181, 2017.
- [7] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, “Large language models for software engineering: Survey and open problems,” in *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, pp. 31–53, IEEE, 2023.
- [8] Z. Zheng, K. Ning, Y. Wang, J. Zhang, D. Zheng, M. Ye, and J. Chen, “A survey of large language models for code: Evolution, benchmarking, and future trends,” *arXiv preprint arXiv:2311.10372*, 2023.

7 Appendix

7.1 Teammates' Contribution

Teammate	Week 1	Week 2	Week 3	Week 4
Mihir Sane	4h	7h	10h	7h
Aryan Solanki	4h	7h	10h	7h
Sujit Maniya	4h	7h	10h	7h
Jenish Jai	4h	7h	10h	7h
Indudhara S	4h	7h	10h	7h

Table 3: Teammates' contribution summary

7.2 Shell Scripts

The complete workflow for both weeks was automated using two Bash shell scripts. The first script, `run_week1_arcade.sh`, sequentially executes all dependency extraction and clustering steps running the ARCADE JavaParser to generate the master RSF file, filtering it to the Capacity Scheduler package, and applying ACDC, WCA, and Limbo clustering producing all required output files in a single run. The second script, `run_week2_metrics_group10.sh`, automates the pairwise evaluation by running both the A2a and Cvg metric tools across all three algorithm pairs (ACDC vs Limbo, ACDC vs WCA, Limbo vs WCA) and saving the results to a consolidated output file for analysis. Both scripts include input validation and are fully reproducible given the same input JAR and ARCADE tool binaries. `run_week3_arc.sh` compares the Week 3 ARC clustering output against the Week 1 ACDC, WCA, and LIMBO outputs.

7.3 Prompt Templates Used for Code Analysis

For completeness and reproducibility, the most important prompt templates used to guide the large language model throughout the analysis are provided below.

7.3.1 Default File-Level Prompt

You are an expert software architect. Your task is to analyze the code and provide:

1. Key Functionality:
A concise, high-level overview of what this code does.
2. Core Logic:
A step-by-step explanation of the main algorithm or business logic.
3. Inputs/Outputs:
A description of the primary inputs the code takes and the

outputs it produces.

4. Dependencies:

A list of key classes, interfaces, or external libraries that this code depends on.

7.3.2 Directory-Based Final Architecture Prompt

Based on the following high-level summaries of different sub-modules, generate a final, comprehensive architectural summary.

Describe the overall purpose of the entire module, how the sub-modules interact, and the key responsibilities of each part.

7.3.3 Cluster-Based File-Level Prompt

You are an expert software architect.

Analyze the provided Java source file and produce a concise file-level semantic summary.

Your summary must include:

1. Key Functionality:

What this file/class mainly does.

2. Core Logic:

The important algorithm, control flow, or business logic.

3. Inputs/Outputs:

Main inputs, outputs, state changes, or side effects.

4. Dependencies:

Important classes, interfaces, libraries, frameworks, or Hadoop/YARN APIs used.

Keep the summary concise but specific.

Return only the summary text.

7.3.4 Cluster-Based Cluster-Level Prompt

You are a software architect.

You are given file-level summaries for files belonging to one recovered software cluster.

Do not describe every file separately.

Instead, synthesize an architectural cluster-level result.

Generate:

1. A short architectural title.
2. A concise architectural description under 150 words.

The description must explicitly include:

- Main components in the cluster.
- How the components interact.
- Relevant quality attributes such as maintainability, scalability, reliability, configurability, performance, or security.
- Technologies, frameworks, languages, or libraries used.
- The overall architectural role of the cluster.

Return only:

Title:

<generated title>

Description:

<generated description>

7.4 Output Example of file summary:

`AbsoluteResourceCapacityCalculator.txt`

The following example illustrates the file-level semantic summary generated by the proposed approach for the source file `AbsoluteResourceCapacityCalculator.java`.

Here is a concise file-level semantic summary:

Key Functionality:

This Java class, `AbsoluteResourceCapacityCalculator`, calculates resource capacities for YARN (Yet Another Resource Negotiator) queues with absolute capacity type, ensuring efficient resource allocation in a Hadoop cluster.

Core Logic:

- * Normalizes resource ratios for parent queues with absolute capacity type, considering children's configured resources and effective minimum resources.
- * Calculates minimum and maximum resource capacities for queues based on normalized ratios, remaining resources, and current capacity entries.
- * Updates queue capacities after calculations, handling potential downscaling warnings.

Inputs/Outputs:

Inputs:

- ResourceCalculationDriver (provides context for resource calculations)
- CalculationContext (offers resource names, current capacity entries, etc.)
- CSQueue (represents the queue being calculated for)

Outputs/State Changes:

- Updated queue capacities
- Normalized resource ratios (stored in ResourceCalculationDriver)
- Potential downscaling warnings (added to UpdateContext)

Dependencies:

Classes/Interfaces:

- AbstractQueueCapacityCalculator (parent class)
- ResourceCalculationDriver
- CalculationContext
- CSQueue
- QueueCapacityVector
- ResourceVector

Libraries/Frameworks:

- Apache Hadoop YARN
- Apache Hadoop common utilities (e.g., UnitsConversionUtil)

7.5 Cluster-based hierarchical summarization:

Cluster ID:

ARC_07

Files:

CSQueuePreemptionSettings.java
preemption/KillableContainer.java
preemption/PreemptableQueue.java
preemption/PreemptionManager.java

Title:

YARN Capacity Scheduler Preemption Management Cluster

Description:

This architectural cluster, part of the YARN (Yet Another Resource Negotiator) Capacity Scheduler, manages preemption for queues, enabling efficient resource allocation and deallocation.

Main Components:

- **CSQueuePreemptionSettings:** Evaluates preemption settings for queues.
- **KillableContainer:** Represents preemptable containers with metadata.
- **PreemptableQueue:** Manages preemptable resources and containers for a queue.
- **PreemptionManager:** Oversees preemptable resources across queues, maintaining a hierarchical structure.

Component Interactions:

- **PreemptionManager** utilizes **CSQueuePreemptionSettings** for evaluating preemption settings.
- **PreemptableQueue** interacts with **KillableContainer** for resource tracking.
- **PreemptionManager** coordinates with **PreemptableQueue** instances for hierarchical management.

Quality Attributes:

- **Maintainability:** Modular design with clear component responsibilities.
- **Scalability:** Hierarchical queue structure and concurrent access (via **ReentrantReadWriteLock**) support large-scale resource management.
- **Reliability:** Thread-safe access ensures consistent state.
- **Configurability:** Preemption settings can be dynamically evaluated and updated.

Technologies, Frameworks, Languages, and Libraries:

- Apache Hadoop YARN
- Apache Hadoop Configuration
- Java (with concurrency utilities)

7.6 Directory based leaf node (UserWeights) summarization:**Analysis of the UserWeights Class****1. Key Functionality:****High-Level Overview:**

The **UserWeights** class manages a collection of user-specific weights for a queue in a resource management system (Apache Hadoop YARN). It provides methods to create, retrieve, validate, and combine user weights.

2. Core Logic:**Step-by-Step Explanation of the Main Algorithm/Business Logic:****1. Initialization:**

- The class can be initialized in two ways:
 - **Empty:** `createEmpty()` returns an instance with an empty weight map.
 - **From Configuration:** `createByConfig()` populates the weight map based on configuration properties for a specified queue.

2. Weight Retrieval:

- `getUser(String userName)` returns the weight for a given user, defaulting to `DEFAULT_WEIGHT` (1.0F) if the user is not found.

3. Validation:

- `validateForLeafQueue(float queueUserLimit, String queuePath)` checks all user weights against a specified limit (`100 / queueUserLimit`) for a leaf queue, throwing an `IOException` for any invalid weights.

4. Weight Combination:

- `addFrom(UserWeights addFrom)` merges the weights from another `UserWeights` instance into the current one.

3. Inputs/Outputs:

Primary Inputs and Outputs:

- **Inputs:**
 - `userName` (String) for weight retrieval.
 - `CapacitySchedulerConfiguration` and `ConfigurationProperties` for initialization from configuration.
 - `queueUserLimit` (float) and `queuePath` (String) for validation.
 - Another `UserWeights` instance for weight combination.
- **Outputs:**
 - **Weight Values** (float) for a given user.
 - **Validation Result** (throws `IOException` on failure).
 - **Updated UserWeights Instance** after combining weights.

4. Dependencies:

Key Classes, Interfaces, or External Libraries:

- **Apache Hadoop YARN Libraries:**
 - `org.apache.hadoop.yarn.server.resourcemanager.scheduler.capacity.CapacitySchedulerConfiguration`
 - `org.apache.hadoop.yarn.server.resourcemanager.scheduler.capacity.QueuePrefixes`
- **Java Standard Library:**

- `java.io.IOException`
- `java.util.HashMap`
- `java.util.Map`
- `java.util.regex`

7.7 Directory based branch node (allocator) summarization:

Concise Summary of the allocator Sub-module's Role and Functionality

Role:

The `allocator` sub-module is a crucial component of the Apache Hadoop YARN (Yet Another Resource Negotiator) Resource Manager, specifically designed for capacity scheduling. Its primary role is to manage the allocation and deallocation of containers for YARN applications, ensuring efficient resource utilization within the cluster.

Key Functionality:

1. **Container Allocation Management:** Decides how to allocate containers based on application resource requests, considering factors like locality (node, rack, off-switch), resource availability, and scheduling modes.
2. **State Management:** Tracks and updates the state of allocation requests (e.g., allocated, reserved, skipped) to inform subsequent scheduling decisions.
3. **Resource Utilization Optimization:** Aims to optimize resource allocation by evaluating requests against resource limits, cluster availability, and scheduling opportunities.
4. **Extensibility:** Provides an abstract foundation (`AbstractContainerAllocator`) for developing custom allocation logic, with a concrete implementation (`RegularContainerAllocator`) for standard capacity scheduling scenarios.
5. **Delegation and Abstraction:** Employs a delegation model (`ContainerAllocator` delegating to `RegularContainerAllocator`) to abstract the complexity of allocation logic, enhancing maintainability and flexibility.

Core Components and Their Responsibilities:

- **`AllocationState.java`:** Defines enumeration values for representing allocation states.
- **`ContainerAllocation.java`:** Represents a single container allocation request, managing its state and metadata.
- **`AbstractContainerAllocator.java`:** Provides an abstract base for container allocation logic, outlining the allocation process.
- **`RegularContainerAllocator.java`:** Implements the standard container allocation logic for capacity scheduling, considering locality and resource availability.

- **ContainerAllocator.java:** Delegates container allocation to `RegularContainerAllocator`, abstracting the allocation complexity.

7.8 Directory based summarization of final architecture:

Comprehensive Architectural Summary

Overall Purpose:

The entire module, part of the Apache Hadoop YARN (Yet Another Resource Negotiator) project, is designed to manage resource allocation, queue configuration, and application lifecycle within a distributed computing environment, specifically focusing on the Capacity Scheduler. Its primary goal is to ensure efficient resource utilization, provide flexible queue management, and support various scheduling policies.

Sub-Module Interactions and Key Responsibilities:

1. **top_level** (Capacity Scheduler Core)
 - **Responsibility:** Orchestrates the overall resource management and scheduling process.
 - **Interactions:** Coordinates with all sub-modules to ensure cohesive functionality.
2. **allocator** (Container Allocation Manager)
 - **Responsibility:** Manages container allocation and deallocation, optimizing resource utilization.
 - **Interactions:**
 - Receives resource requests from **top_level**.
 - Collaborates with **preemption** for container termination (if necessary).
3. **conf** (Configuration Manager)
 - **Responsibility:** Handles configuration storage, access control, and versioning for the Capacity Scheduler.
 - **Interactions:**
 - Provides configuration data to **top_level** and other sub-modules.
 - Validates updates from **top_level** and **queuemanagement**.
4. **placement** (Placement Decision Support)
 - **Responsibility:** Translates predefined rules into actionable configurations for resource allocation.
 - **Interactions:**
 - Supplies placement decisions to **allocator**.

- Receives rule updates from **top_level** (indirectly, through configuration changes).

5. **policy** (Queue Ordering Policy Manager)

- **Responsibility:** Determines the allocation order of queues based on predefined strategies.
- **Interactions:**
 - Provides queue ordering to **allocator**.
 - Integrates with **top_level** for queue management.

6. **preemption** (Resource Preemption Manager)

- **Responsibility:** Identifies and terminates lower-priority containers to satisfy higher-priority applications.
- **Interactions:**
 - Collaborates with **allocator** for container termination.
 - Updates **queuemanagement** on resource availability.

7. **queuemanagement** (Auto-Created Leaf Queue Manager)

- **Responsibility:** Dynamically adjusts capacities of auto-created leaf queues based on application demand and node labels.
- **Interactions:**
 - Receives resource availability updates from **preemption**.
 - Coordinates with **conf** for configuration validation.